FOOT5

DOSSIER TECHNIQUE COMPLET

Maison des Ligues de Lorraine

Projet de Soutenance -- BTS SIO SLAM

HAKIM RAYANE

Mars 2026

SOMMAIRE

DOC 01 Tableau de Synthese Comparative 3

DOC 02 Dossier Principal -- Partie Commune 7

DOC 03 Application Legere -- Web Client 16

DOC 04 Application Lourde -- Admin Desktop 26

DOC 05 Code Source -- Liens GitHub 37

DOC 01

Tableau de Synthèse -- Projet PPE Foot5

Étudiant : Hakim Rayane -- BTS SIO SLAM

Date : Mars 2026

Phrase clé du projet :

"Le projet Foot5 repose sur une architecture composée de deux applications complémentaires : une application web destinée aux clients et une application lourde destinée à l'administration, partageant une base de données commune."

Vue d'Ensemble des Deux Applications

Critère	Application Légère (Web)	Application Lourde (Admin)
Nom	Foot5 -- Extension Web	Football Manager 5v5 -- Desktop
Type	Application web (SPA)	Application desktop
Public cible	Clients / utilisateurs finaux	Administrateurs / gestionnaires
Rôle	Interface de réservation côté client	Gestion complète de la plateforme
Déploiement	Navigateur web (localhost:3000)	Installé sur poste admin
GitHub	projet-ppe-foot5-web	projet-ppe-exam

Technologies

Composant	Application Légère (Web)	Application Lourde (Admin)
Frontend	React 18 + TypeScript + Tailwind CSS	PySide6 (Qt6) -- interface native
Backend	FastAPI (Python)	-- (accès direct BDD via SQLAlchemy)
ORM	SQLAlchemy 2.0	SQLAlchemy 2.0
Base de données	MySQL foot5 (partagée)	MySQL foot5 (partagée)
Authentification	JWT (JSON Web Token)	Session locale (bcrypt)
Hachage mdp	bcrypt (12 rounds)	bcrypt (12 rounds)
Tests	pytest + requests (API)	unittest + pytest (unitaires/intégration)
Langage principal	TypeScript / Python	Python 3.8+

Fonctionnalités

Fonctionnalité	App Légère (Web)	App Lourde (Admin)
Connexion / Authentification	[OK] Via JWT	[OK] Via session bcrypt
Inscription utilisateur	[OK]	[X] (créé par admin)
Réserver un terrain	[OK]	[OK]
Voir ses réservations	[OK] (Mon Espace)	[OK] (toutes)
Annuler une réservation	[OK]	[OK]
Confirmer une réservation	[X]	[OK] (admin/superadmin)
Gestion des utilisateurs (CRUD)	[X]	[OK]
Gestion des terrains (CRUD)	[X]	[OK]
Activer / désactiver terrain	[X]	[OK]
Calendrier des réservations	[X]	[OK] (calendrier interactif)
Tableau de bord statistiques	[X]	[OK]
Détection conflits horaires	[OK] (API)	[OK] (BDD + service)
Quota réservations hebdo	[OK] (2/semaine max)	[X] (admin sans limite)
Calcul coût total	[OK] (tarif horaire)	[OK]
Gestion des rôles	Lecture (user/admin)	Écriture (superadmin/admin/user)

Architecture Partagée

```
Clients (navigateur)          Admins (poste)
        |                           |
        v                           v
 App Légère (Web)         App Lourde (Desktop)
 React + FastAPI          Python + PySide6
        |                           |
        +----------+----------------+
                   |
                   v
          Base MySQL "foot5"
          +--------------+
          | users        |
          | terrains     |
```

```
          | reservations |
          +--------------+
```

Rôles Utilisateurs

Rôle	App Web	App Desktop
user	Réserve, consulte son espace	Limité (pas d'accès admin)
admin	Voir toutes les réservations, confirmer	Gestion réservations + terrains
superadmin	--	Accès total : users + terrains + réservations

Points Forts du Projet

Aspect	Détail
Cohérence	BDD unique partagée entre les deux apps
Complémentarité	Client léger (web) + outil admin puissant
Sécurité	bcrypt 12 rounds + JWT + validation métier
Architecture	MVC côté desktop, REST API côté web
Qualité code	87% couverture tests, PEP8, TypeScript strict
Performance	< 2 secondes pour toutes les opérations clés

Hakim Rayane -- BTS SIO SLAM -- PPE Foot5 -- Mars 2026

DOC 02

Dossier Principal -- Partie Commune

Projet PPE Foot5 -- BTS SIO SLAM

Étudiant : Hakim Rayane

Date : Mars 2026

Version : 2.1

1. Contexte Global

1.1 Présentation de l'Entreprise / Organisme

Le projet Foot5 s'inscrit dans le contexte de la Maison des Ligues de Lorraine (M2L), un établissement sportif gérant plusieurs infrastructures dont des terrains de football à cinq (5v5).

Problématique initiale :

- Gestion manuelle des réservations (agenda papier, appels téléphoniques)
- Conflits de créneaux non détectés
- Manque de visibilité pour les clients
- Charge administrative élevée pour le personnel

1.2 Besoin Global

L'organisation a besoin d'un système informatisé complet couvrant deux niveaux d'utilisation :

Besoin	Description
Côté client	Permettre aux joueurs de consulter les terrains et réserver un créneau en ligne
Côté admin	Donner aux gestionnaires un outil puissant de gestion des réservations, utilisateurs et terrains
Cohérence des données	S'assurer que les deux systèmes partagent les mêmes informations en temps réel

1.3 Solution Retenue

La solution développée repose sur deux applications complémentaires qui partagent une base de données MySQL commune :

1. Application légère (web) -- accessible depuis n'importe quel navigateur, destinée aux clients
2. Application lourde (desktop) -- installée sur le poste des gestionnaires, interface native complète

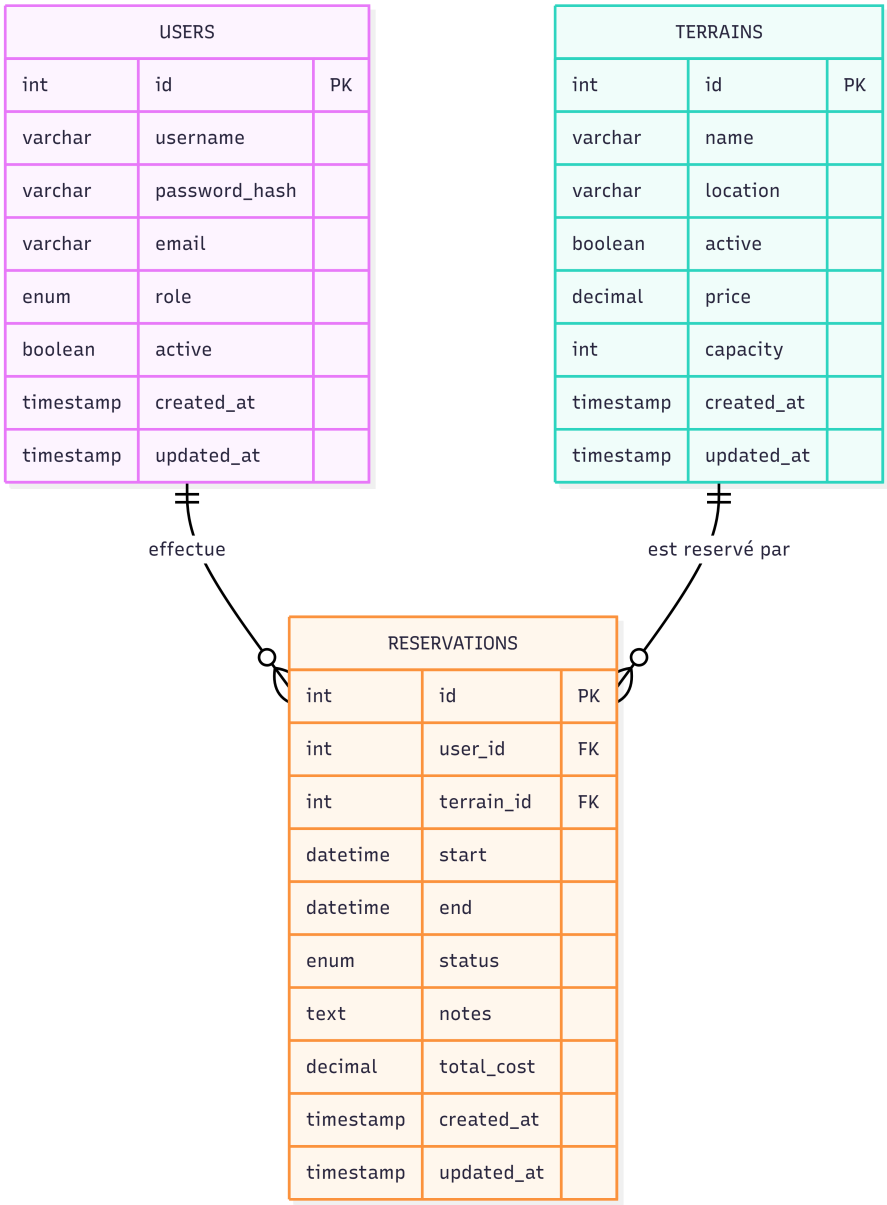
> *"Le projet Foot5 repose sur une architecture composée de deux applications complémentaires : une application web destinée aux clients et une application lourde destinée à l'administration, partageant une base de données commune."*

---

2. Base de Données Commune

> *Point clé : La base de données foot5 est partagée entre l'application web et l'application desktop. Toute modification effectuée dans l'une est immédiatement visible dans l'autre.*

2.1 Modèle Conceptuel de Données (MCD)



MCD — Modèle Conceptuel de Données Merise

Relations :

- Un UTILISATEUR peut effectuer de 0 à n RÉSERVATIONS
- Un TERRAIN peut être concerné par 0 à n RÉSERVATIONS
- Une RÉSERVATION est effectuée par exactement 1 UTILISATEUR pour exactement 1 TERRAIN

2.2 Modèle Logique de Données (MLD)

```
users (
    id              : INTEGER        PRIMARY KEY AUTO_INCREMENT,
    username        : VARCHAR(50)    UNIQUE NOT NULL,
    password_hash   : VARCHAR(255)   NOT NULL,
    email           : VARCHAR(120)   UNIQUE NOT NULL,
    role            : ENUM('user', 'admin', 'superadmin') NOT NULL DEFAULT 'user',
    active          : BOOLEAN        NOT NULL DEFAULT TRUE,
```

```
    created_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP,
    updated_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
)

terrains (
    id              : INTEGER        PRIMARY KEY AUTO_INCREMENT,
    name            : VARCHAR(100)   NOT NULL,
    location        : VARCHAR(200),
    active          : BOOLEAN        DEFAULT TRUE,
    price           : DECIMAL(10,2)  DEFAULT 0.00,    ← tarif horaire (ajout mars 2026)
    capacity        : INTEGER        DEFAULT 10,       ← ex: 10 = 5v5 (ajout mars 2026)
    created_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP,
    updated_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
)

reservations (
    id              : INTEGER        PRIMARY KEY AUTO_INCREMENT,
    user_id         : INTEGER        NOT NULL → FK → users(id) ON DELETE CASCADE,
    terrain_id      : INTEGER        NOT NULL → FK → terrains(id) ON DELETE CASCADE,
    start           : DATETIME       NOT NULL,
    end             : DATETIME       NOT NULL,
    status          : ENUM('pending', 'confirmed', 'cancelled') DEFAULT 'pending',
    notes           : TEXT,
    total_cost      : DECIMAL(10,2)  DEFAULT 0.00,    ← calcul auto à la création
    created_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP,
    updated_at      : TIMESTAMP      DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id)    REFERENCES users(id)    ON DELETE CASCADE,
    FOREIGN KEY (terrain_id) REFERENCES terrains(id) ON DELETE CASCADE,
    UNIQUE KEY unique_terrain_time (terrain_id, start, end)
)
```

2.3 Script SQL Complet

```
-- ============================================================
-- Foot5 -- Base de données commune
-- Compatible MySQL 8.0+ / MariaDB 10.5+
-- Partagée entre l'application web et l'application desktop
-- ============================================================

CREATE DATABASE IF NOT EXISTS foot5
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE foot5;

-- Table des utilisateurs
CREATE TABLE IF NOT EXISTS users (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    username      VARCHAR(50)  UNIQUE NOT NULL,
```

```
    password_hash VARCHAR(255) NOT NULL,
    email         VARCHAR(120) UNIQUE NOT NULL,
    role          ENUM('user', 'admin', 'superadmin') NOT NULL DEFAULT 'user',
    active        BOOLEAN DEFAULT TRUE,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_username (username),
    INDEX idx_email    (email),
    INDEX idx_role     (role),
    INDEX idx_active   (active)
) ENGINE=InnoDB;

-- Table des terrains
CREATE TABLE IF NOT EXISTS terrains (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    location   VARCHAR(200),
    active     BOOLEAN DEFAULT TRUE,
    price      DECIMAL(10,2) DEFAULT 0.00,
    capacity   INT DEFAULT 10,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX idx_name   (name),
    INDEX idx_active (active)
) ENGINE=InnoDB;

-- Table des réservations
CREATE TABLE IF NOT EXISTS reservations (
    id           INT AUTO_INCREMENT PRIMARY KEY,
    user_id      INT NOT NULL,
    terrain_id   INT NOT NULL,
    start        DATETIME NOT NULL,
    end          DATETIME NOT NULL,
    status       ENUM('pending', 'confirmed', 'cancelled') DEFAULT 'pending',
    notes        TEXT,
    total_cost   DECIMAL(10,2) DEFAULT 0.00,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id)    REFERENCES users(id)    ON DELETE CASCADE,
    FOREIGN KEY (terrain_id) REFERENCES terrains(id) ON DELETE CASCADE,
    INDEX idx_user_id          (user_id),
    INDEX idx_terrain_id       (terrain_id),
    INDEX idx_start_time       (start),
    INDEX idx_status           (status),
    INDEX idx_reservation_period (start, end),
    UNIQUE KEY unique_terrain_time (terrain_id, start, end)
) ENGINE=InnoDB;

-- Vue : détails des réservations
CREATE OR REPLACE VIEW reservation_details AS
SELECT
    r.id, r.start, r.end, r.status, r.notes, r.total_cost,
    u.username, u.email, u.role,
```

```
    t.name AS terrain_name, t.location AS terrain_location,
    t.active AS terrain_active, t.price AS tarif_horaire, t.capacity
FROM reservations r
JOIN users u    ON r.user_id    = u.id
JOIN terrains t ON r.terrain_id = t.id;

-- Vue : statistiques tableau de bord
CREATE OR REPLACE VIEW dashboard_stats AS
SELECT
    (SELECT COUNT(*) FROM users      WHERE active = TRUE) AS active_users,
    (SELECT COUNT(*) FROM terrains   WHERE active = TRUE) AS active_terrains,
    (SELECT COUNT(*) FROM reservations
     WHERE status IN ('pending', 'confirmed')
     AND DATE(start) = CURDATE())                        AS today_reservations,
    (SELECT COUNT(*) FROM reservations
     WHERE status IN ('pending', 'confirmed'))           AS total_active_reservations;
```

2.4 Données de Test

```
-- Données initiales (seed)
INSERT INTO terrains (name, location, active, price, capacity) VALUES
('Terrain A', 'Salle Centrale', 1, 20.00, 10),
('Terrain B', 'Salle Nord',     1, 20.00, 10),
('Terrain C', 'Salle Sud',      1, 15.00, 10),
('Terrain D', 'Salle Est',      1, 20.00, 14),
('Terrain E', 'Salle Ouest',    1, 20.00, 10),
('Terrain F', 'Salle Annexe',   0, 10.00, 10);  -- Inactif (test)

-- Comptes utilisateurs par défaut (mots de passe hachés bcrypt)
-- admin / admin123 | manager / manager123 | user1 / user123
```

2.5 Pourquoi une Base Commune ?

Avantage	Explication
Cohérence des données	Un client qui réserve en ligne est immédiatement visible dans l'app admin
Pas de synchronisation	Pas de risque de désynchronisation ou de doublons
Hachage compatible	bcrypt 12 rounds identique → même table users utilisée par les deux apps
Évolutivité	Ajout de nouvelles colonnes avec DEFAULT → rétrocompatible
Détection de conflits	La contrainte UNIQUE KEY unique_terrain_time est garantie au niveau BDD

2.6 Contraintes d'Intégrité

1. Clés primaires AUTO_INCREMENT NOT NULL sur toutes les tables
2. Clés étrangères ON DELETE CASCADE -- suppression en cascade cohérente
3. UNIQUE sur username et email -- pas de doublons utilisateurs
4. ENUM pour les rôles (user/admin/superadmin) et statuts (pending/confirmed/cancelled)
5. Contrainte temporelle CHECK (end > start) -- une réservation doit avoir une durée positive
6. Anti-conflit UNIQUE KEY unique_terrain_time (terrain_id, start, end) -- pas de double réservation

---

3. Architecture Globale

3.1 Vue d'Ensemble

```
                        +--------------------------------+
                        |        BASE DE DONNÉES         |
                        |         MySQL "foot5"          |
                        |  +--------------------------+  |
                        |  |  users                   |  |
                        |  |  terrains                |  |
                        |  |  reservations            |  |
                        |  +--------------------------+  |
                        +-----------+--------------------+
                                    |
              +--------------------+--------------------+
              |                                         |
              v                                         v
+------------------------+          +--------------------------+
```

```
|   APPLICATION WEB       |           |   APPLICATION DESKTOP      |
|   (App Légère)          |           |   (App Lourde)             |
|                         |           |                            |
|  Frontend               |           |  PySide6 (Qt6)             |
|  React + TypeScript     |           |  Interface native          |
|  Tailwind CSS           |           |  Material Design           |
|          |              |           |          |                 |
|  Backend API            |           |  SQLAlchemy                |
|  FastAPI (Python)       |           |  (accès direct BDD)        |
|  JWT Auth               |           |  bcrypt + sessions         |
+-------------------------+          +---------------------------+
       Clients/Joueurs                      Administrateurs
     (navigateur web)                     (poste de travail)
```

3.2 Flux de Données

Scénario : Un client réserve un terrain en ligne

1. Le client se connecte sur l'app web → API vérifie ses identifiants dans users
2. Il sélectionne un terrain → API récupère les données depuis terrains
3. Il choisit un créneau → API vérifie l'absence de conflits dans reservations
4. Il confirme → réservation créée dans reservations avec statut pending
5. L'admin ouvre l'app desktop → voit la réservation dans son tableau de bord
6. L'admin confirme → statut passe à confirmed dans la BDD
7. La mise à jour est immédiatement visible côté web (même BDD)

3.3 Sécurité Transversale

Mécanisme	App Web	App Desktop	Description
Hachage mdp	bcrypt (12 rounds)	bcrypt (12 rounds)	Identique, table partagée
Auth	JWT (30 min)	Session applicative	Adapté à chaque contexte
Rôles	Vérification sur API	Vérification controller	RBAC (Role-Based Access Control)
Validation	Pydantic (schémas)	SQLAlchemy + Python	Validation des entrées
Anti-duplication	Quota 2/semaine	Contrainte UNIQUE BDD	Double protection

3.4 Technologies Transversales

Technologie	Usage dans les deux apps
MySQL	Base de données unique partagée
SQLAlchemy	ORM Python utilisé côté backend web ET côté desktop
bcrypt	Hachage des mots de passe (même algorithme, mêmes rounds)
Python	Langage backend web (FastAPI) + langage desktop (PySide6)

Hakim Rayane -- BTS SIO SLAM -- PPE Foot5 -- Mars 2026

DOC 03

Application Légère -- Web Client

Foot5 Extension Web -- BTS SIO SLAM

Étudiant : Hakim Rayane

Date : Mars 2026

Dépôt : https://github.com/warkozz/projet-ppe-foot5-web

1. Présentation

L'application web Foot5 est l'interface cliente du système de gestion de terrains de football 5v5. Elle permet à n'importe quel joueur d'accéder à la plateforme depuis son navigateur, sans installation, et de réserver un créneau en quelques clics.

Caractéristiques principales

- Interface responsive accessible depuis tout navigateur
- Aucune installation requise côté client
- Synchronisation en temps réel avec la base de données partagée
- Inscription en ligne autonome
- Calcul automatique du coût de la réservation

Statut du projet

Composant	Statut
Backend API (FastAPI)	Terminé [OK]
Frontend React	Terminé [OK]
Tests API	Terminés [OK]
Documentation	Complète [OK]

2. Fonctionnalités

2.1 Authentification et Gestion du Profil

Fonctionnalité	Description
Inscription	Création d'un compte avec username, email, mot de passe
Connexion	Authentification JWT, token valide 30 minutes
Mon Profil	Consultation et modification des informations personnelles
Changer le mot de passe	Modification sécurisée avec vérification de l'ancien mot de passe
Renouveler le token	Renouvellement automatique de session

Endpoints d'authentification :

```
POST /api/auth/register    -- Inscription
POST /api/auth/login       -- Connexion
GET  /api/auth/me          -- Profil utilisateur
PUT  /api/auth/me          -- Modifier profil
PUT  /api/auth/me/password -- Changer mot de passe
POST /api/auth/refresh     -- Renouveler token
```

2.2 Réservation de Terrain

Le parcours de réservation se déroule en 3 étapes :

Étape 1 -- Choisir un terrain

- Liste des terrains actifs avec nom, localisation, tarif/h, format (5v5, 7v7...)
- Affichage du prix et de la capacité dynamiquement

Étape 2 -- Sélectionner un créneau

- Vérification en temps réel des créneaux disponibles
- Affichage du planning complet du terrain pour le jour choisi
- Détection automatique des conflits

Étape 3 -- Confirmer

- Affichage du coût total calculé automatiquement (durée × tarif horaire)
- Confirmation avec possibilité d'ajouter une note
- Création de la réservation avec statut pending

Règle métier : quota de 2 réservations maximum par semaine (7 jours glissants, configurable)

Endpoints de réservation :

```
GET    /api/reservations                              -- Mes réservations
POST   /api/reservations                              -- Créer (quota vérifié, coût calculé)
GET    /api/reservations/{id}                         -- Détail
PUT    /api/reservations/{id}                         -- Modifier
DELETE /api/reservations/{id}                         -- Annuler
GET    /api/reservations/availability/slots/{id}?date -- Créneaux disponibles
GET    /api/reservations/availability/terrain/{id}?date -- Planning complet
```

2.3 Mon Espace (Historique)

- Affichage de toutes ses réservations
- Filtres par statut : En attente / Confirmée / Annulée
- Tri par date
- Codes couleurs : [?] En attente, [V] Confirmée, [X] Annulée

2.4 Gestion des Terrains (rôle admin)

- Liste des terrains depuis l'API (public)
- Création, modification, suppression (admin)
- Activation / désactivation d'un terrain
- Consultation du format et du tarif

Endpoints terrain :

```
GET    /api/terrains             -- Liste des terrains actifs (public)
GET    /api/terrains/{id}       -- Détail d'un terrain
POST   /api/terrains            -- Créer (admin)
PUT    /api/terrains/{id}       -- Modifier (admin)
DELETE /api/terrains/{id}       -- Supprimer (admin)
PATCH  /api/terrains/{id}/toggle-active -- Activer/Désactiver (admin)
```

---

3. Technologies

3.1 Stack Technique

```
+-----------------------------------------------+
|                  FRONTEND                     |
|  React 18 + TypeScript                        |
|  Tailwind CSS (responsive design)             |
|  Axios (appels HTTP vers l'API)               |
|  React Router DOM (navigation)                |
|  AuthContext (gestion état d'authentification)|
+-------------------+---------------------------+
                    | HTTP/REST (JSON)
                    v
+-----------------------------------------------+
|                   BACKEND                     |
|  FastAPI (Python) -- port 8000                 |
|  SQLAlchemy 2.0 (ORM)                         |
|  Pydantic (validation des données)            |
|  python-jose (JWT)                            |
|  bcrypt (hachage mots de passe)               |
|  pytest + requests (tests)                    |
+-------------------+---------------------------+
                    | SQLAlchemy
                    v
         Base MySQL "foot5" (partagée)
```

3.2 Versions et Dépendances

Backend (requirements.txt) :

```
fastapi
uvicorn
sqlalchemy
pymysql
python-jose[cryptography]
passlib[bcrypt]
pydantic
python-dotenv
pytest
requests
```

Frontend (package.json) :

```
{
  "dependencies": {
    "react": "^18.x",
    "react-dom": "^18.x",
    "typescript": "^5.x",
    "react-router-dom": "^6.x",
```

```
    "axios": "^1.x",
    "tailwindcss": "^3.x"
  }
}
```

3.3 Architecture du Projet

```
projet-ppe-foot5-web/
+-- backend/
|   +-- main.py                   # Point d'entrée FastAPI
|   +-- app/
|   |   +-- models/               # User, Terrain, Reservation (SQLAlchemy)
|   |   +-- routes/               # auth.py, terrains.py, reservations.py
|   |   +-- schemas/              # Validation Pydantic (entrées/sorties)
|   |   +-- services/             # Logique métier (quota, coût, conflits)
|   |   +-- utils/                # hashing.py, jwt.py, config.py
|   +-- test_api.py               # Tests fonctionnels API
|   +-- test_logic.py             # Tests logique métier
|   +-- requirements.txt
+-- frontend/
    +-- src/
        +-- pages/                # Accueil, Terrains, Réservation, Mon Espace, Profil
        +-- components/           # Navigation, Footer, Button, Badge, Alert...
        +-- contexts/             # AuthContext (état utilisateur global)
        +-- services/             # api.ts (toutes les fonctions Axios)
        +-- types/                # Interfaces TypeScript
```

3.4 Pattern Architectural

L'application web suit un pattern REST API + SPA :

- Le frontend React est un single-page application qui consomme l'API
- Le backend FastAPI expose des endpoints REST stateless
- La communication est JSON over HTTP
- L'authentification est stateless via JWT

---

4. Interfaces (Description)

4.0 Charte Graphique

Élément	Valeur
Couleur primaire	#10B981 (Vert Emeraude -- Tailwind emerald-500)
Couleur secondaire	#059669 (Vert Foncé -- Tailwind emerald-600)
Couleur accent	#34D399 (Vert Clair -- Tailwind emerald-400)
Fond principal	#FFFFFF (Blanc)
Fond secondaire	#F9FAFB (Gris très clair -- Tailwind gray-50)
Texte principal	#111827 (Quasi-noir -- Tailwind gray-900)
Texte secondaire	#6B7280 (Gris -- Tailwind gray-500)
Erreur / Annulé	#EF4444 (Rouge -- Tailwind red-500)
Succès / Confirmé	#10B981 (Vert -- Tailwind emerald-500)
Attente	#F59E0B (Ambre -- Tailwind amber-500)
Typographie	Inter / system-ui -- 14-20px
Border-radius	8px (cards), 6px (boutons), 4px (inputs)
Framework CSS	Tailwind CSS 3.x (utility-first)

4.1 Page d'Accueil

- Présentation du service Foot5
- Boutons "Réserver" et "Se connecter"
- Liste des terrains disponibles (public)

4.2 Page de Connexion / Inscription

- Formulaire de connexion avec gestion des erreurs
- Lien vers l'inscription
- Messages d'erreur contextuels (identifiants incorrects, email déjà utilisé...)

4.3 Page Terrains

- Cards affichant chaque terrain :
  - Nom et localisation
  - Format (ex: "5v5 -- 10 joueurs")
  - Tarif horaire (ex: "20€/h")
  - Statut (Actif / Inactif)
  - Bouton "Réserver"

4.4 Parcours de Réservation (3 étapes)

1. Choix du terrain : liste visuelle avec format et prix
2. Choix du créneau : sélection date + heure de début + durée + affichage coût estimé
3. Confirmation : récapitulatif avec coût total + champ notes + bouton confirmer

4.5 Mon Espace

- Tableau de l'historique des réservations
- Filtres par statut + tri par date
- Affichage du coût total de chaque réservation
- Bouton d'annulation si réservation en attente

4.6 Mon Profil

- Affichage des informations du compte
- Formulaire de modification (email, username)
- Formulaire de changement de mot de passe

4.7 Pages Légales

Pages CGU, Politique de confidentialité, Mentions légales

---

5. Sécurité

5.1 Authentification JWT

```
Client                    Serveur FastAPI
  |                             |
  |  POST /api/auth/login       |
  | ---------------------------->|
  |  {username, password}       |
  |                             | → vérifie bcrypt(password, hash)
  |  {access_token: "eyJ..."}   |
  | <----------------------------|
  |                             |
  |  GET /api/reservations      |
  |  Authorization: Bearer eyJ...|
  | ---------------------------->|
  |                             | → valide JWT (expiration, signature)
  |  [{id:1, terrain:...}]      |
  | <----------------------------|
```

- Token JWT signé (algorithme HS256)

- Expiration : 30 minutes
- Transmission via header Authorization: Bearer <token>

5.2 Contrôle d'Accès (RBAC)

Route	Public	User	Admin
GET /api/terrains	[OK]	[OK]	[OK]
POST /api/reservations	[X]	[OK]	[OK]
GET /api/reservations	[X]	Ses resas	Toutes
PATCH .../confirm	[X]	[X]	[OK]
POST /api/terrains	[X]	[X]	[OK]

5.3 Validation des Données

- Pydantic valide toutes les entrées API (types, longueurs, formats)
- Validation email : regex RFC 5322
- Validation du mot de passe : longueur minimum
- Validation temporelle : end > start obligatoire
- Quota : maximum 2 réservations par semaine (7 jours glissants)

5.4 Protection contre les Conflits

```
# Détection de conflit (côté service)
def check_availability(terrain_id, start, end):
    conflicts = db.query(Reservation).filter(
        Reservation.terrain_id == terrain_id,
        Reservation.status.in_(['pending', 'confirmed']),
        Reservation.start < end,
        Reservation.end > start
    ).count()
    return conflicts == 0
```

---

6. Tests

6.1 Tests API (test_api.py)

Test	Description	Résultat
test_register	Inscription nouvel utilisateur	[OK]
test_login	Connexion avec identifiants valides	[OK]
test_login_invalid	Connexion avec mauvais mdp	[OK] (401)
test_get_profile	Récupérer profil avec token	[OK]
test_get_terrains	Liste des terrains (public)	[OK]
test_create_reservation	Créer une réservation	[OK]
test_conflict_detection	Conflit sur même terrain/créneau	[OK] (409)
test_quota_limit	Dépassement quota 2/semaine	[OK] (429)

6.2 Tests Logique Métier (test_logic.py)

Test	Description	Résultat
test_total_cost_calculation	Coût = durée × tarif	[OK]
test_time_overlap_detection	Détection chevauchement	[OK]
test_weekly_quota_logic	Comptage réservations 7j glissants	[OK]
test_bcrypt_hash_compatible	Hash compatible app desktop	[OK]

6.3 Documentation API Interactive

FastAPI génère automatiquement une documentation Swagger accessible sur :

```
http://localhost:8000/docs        ← Interface Swagger UI
http://localhost:8000/redoc       ← Interface ReDoc
```

7. Installation et Démarrage

Prérequis

- Python 3.8+
- Node.js 18+ et npm
- MySQL avec la base foot5 initialisée

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate         # Windows
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Frontend

```
cd frontend
npm install
npm start
# → http://localhost:3000
```

Configuration (.env)

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=foot5
DB_USER=root
DB_PASSWORD=
JWT_SECRET_KEY=votre-cle-secrete
JWT_ALGORITHM=HS256
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=30
MAX_WEEKLY_RESERVATIONS=2
```

Hakim Rayane -- BTS SIO SLAM -- PPE Foot5 -- Mars 2026

DOC 04

Application Lourde -- Administration Desktop

Football Manager 5v5 -- BTS SIO SLAM

Étudiant : Hakim Rayane

Date : Mars 2026 (initialisé Décembre 2025)

Dépôt : https://github.com/warkozz/projet-ppe-exam

1. Présentation

L'application desktop Football Manager 5v5 est l'outil d'administration du système Foot5. Destinée exclusivement aux gestionnaires et administrateurs, elle offre une interface native puissante permettant de gérer l'intégralité de la plateforme : utilisateurs, terrains et réservations.

Caractéristiques principales

- Interface native (non-navigateur) avec Material Design thème football
- Accès complet à toutes les données de la plateforme
- Gestion multi-rôles : Superadmin, Admin, Utilisateur
- Calendrier interactif pour visualiser les réservations
- Tableau de bord avec statistiques en temps réel

Rôles supportés

Rôle	Droits
Superadmin	Accès total : gestion users + terrains + réservations + création admins
Admin (Gestionnaire)	Gestion des terrains et des réservations (sans gestion des users)
User	Consultation uniquement (accès limité)

2. Fonctionnalités

2.1 Authentification

- Formulaire de connexion sécurisé (username + password)
- Vérification bcrypt (12 rounds) contre la table users
- Affichage du nom et du rôle dans l'interface après connexion
- Messages de bienvenue personnalisés selon le rôle
- Limitation des tentatives : blocage après 5 tentatives échouées
- Navigation par onglets conditionnelle au rôle

2.2 Tableau de Bord (Dashboard)

Vue synthétique de l'état de la plateforme :

- Nombre de terrains actifs
- Nombre de réservations du jour
- Nombre total d'utilisateurs actifs
- Nombre de réservations actives en cours
- Actualisation automatique toutes les 60 secondes

2.3 Gestion des Utilisateurs (Superadmin)

Action	Description
Lister	Tableau : ID, Nom, Email, Rôle, Statut (Actif/Inactif)
Ajouter	Formulaire modal (username, email, mot de passe, rôle)
Modifier	Modification des informations (email, rôle, statut)
Supprimer	Suppression avec confirmation modale (cascade BDD)
Rechercher	Filtre en temps réel par nom ou email

Règle métier : seul le Superadmin peut créer d'autres admins.

2.4 Gestion des Terrains

Action	Description
Lister	Tableau avec nom, localisation, statut (badge couleur), prix, capacité
Ajouter	Formulaire avec nom, localisation, prix/h, capacité de joueurs
Modifier	Mise à jour de toutes les propriétés
Activer / Désactiver	Toggle visuel (vert = actif, rouge = inactif)
Supprimer	Suppression avec confirmation (cascade réservations)

2.5 Calendrier des Réservations

- QCalendarWidget personnalisé avec thème Material
- Points rouges sur les jours ayant des réservations
- Clic sur une date → affichage des réservations du jour
- Ajout direct d'une réservation depuis le calendrier

2.6 Gestion des Réservations

Action	Description
Lister	Tableau : Utilisateur, Terrain, Date/Heure, Durée, Statut, Coût
Ajouter	Formulaire avec sélection user, terrain, créneau + validation conflit
Modifier	Modification des informations de la réservation
Confirmer	Passage du statut pending → confirmed
Annuler	Passage du statut → cancelled
Supprimer	Suppression définitive
Filtrer	Filtres par date, statut, terrain
Détecter les conflits	Validation en temps réel avant création

3. Technologies

3.1 Stack Technique

```
+-----------------------------------------------+
|              APPLICATION DESKTOP              |
|                                               |
|  Interface : PySide6 (Qt6)                    |
|  Thème : Material Design (sombre, vert football)|
|  Architecture : MVC (Model-View-Controller)   |
|                                               |
|  +--------------+  +-------------------------+ |
|  |    Views     |  |      Controllers        | |
|  |  login_view  |  |  auth_controller        | |
|  |  dashboard   |  |  user_controller        | |
|  |  user_view   |  |  terrain_controller     | |
|  |  terrain_view|  |  reservation_controller | |
|  |  resv_view   |  +----------+--------------+ |
|  |  calendar    |             |                |
|  +--------------+             | SQLAlchemy ORM |
|                               v                |
|              +-----------------------------+   |
|              |           Models            |   |
|              |  User - Terrain - Reservation|   |
|              +--------------+--------------+   |
+-----------------------------+-----------------+
                              | MySQL connector
                              v
                  Base MySQL "foot5" (partagée)
```

3.2 Versions et Dépendances

requirements.txt :

```
PySide6>=6.4.0
SQLAlchemy>=2.0.0
PyMySQL>=1.0.0
bcrypt>=4.0.0
python-dotenv>=1.0.0
```

- Python 3.8+
- PySide6 : framework UI Qt6 pour Python
- SQLAlchemy 2.0 : ORM (Object-Relational Mapping)
- PyMySQL : connecteur MySQL pur Python

- bcrypt : hachage sécurisé des mots de passe

3.3 Architecture MVC

```
désktop_app/
+-- hybrid_main.py              # Point d'entrée de l'application
+-- app/
|   +-- config.py               # URL BDD, constantes
|   +-- models/
|   |   +-- db.py               # Engine SQLAlchemy, SessionLocal, Base
|   |   +-- user.py             # Modèle User (ORM)
|   |   +-- terrain.py          # Modèle Terrain (ORM)
|   |   +-- reservation.py      # Modèle Reservation (ORM)
|   +-- controllers/
|   |   +-- auth_controller.py        # Login, rate limiting
|   |   +-- user_controller.py        # CRUD utilisateurs
|   |   +-- terrain_controller.py     # CRUD terrains
|   |   +-- reservation_controller.py # CRUD + validation conflits
|   +-- views/
|   |   +-- login_view.py             # Écran de connexion
|   |   +-- hybrid/
|   |       +-- dashboard_view.py     # Tableau de bord statistiques
|   |       +-- user_view.py          # Gestion des utilisateurs
|   |       +-- terrain_view.py       # Gestion des terrains
|   |       +-- reservation_view.py   # Gestion des réservations
|   |       +-- calendar_view.py      # Calendrier interactif
|   +-- services/
|   |   +-- calendar_service.py       # Logique du calendrier
|   |   +-- cpp_bridge.py             # Pont vers algorithme C++ (optimisation)
|   +-- styles/
|   |   +-- theme.py                  # Thème Material Design
|   +-- utils/
|       +-- hashing.py                # Utilitaires bcrypt
```

3.4 Patterns de Conception Utilisés

Pattern	Utilisation dans l'application
MVC	Architecture globale (Model/View/Controller séparés)
Singleton	Session BDD SQLAlchemy
Observer	Actualisation du dashboard (signal/slot Qt)
Factory	Création des widgets selon le rôle utilisateur
Strategy	Stratégies de validation des réservations
Command	Actions CRUD encapsulées dans les controllers

4. Interfaces

4.1 Charte Graphique

Élément	Valeur
Couleur primaire	#4CAF50 (Vert Football)
Couleur secondaire	#2E7D32 (Vert Foncé)
Couleur accent	#81C784 (Vert Clair)
Fond (Surface)	#1E1E1E (Thème Sombre)
Texte principal	#FFFFFF
Erreur	#F44336 (Rouge)
Typographie	Roboto (Material Design) 14-24px

4.2 Écran de Connexion

```
+----------------------------------------+
|         Football Manager 5v5           |
|          (logo Material Design)        |
|                                        |
|   +--------------------------------+   |
|   |  Nom d'utilisateur             |   |
|   +--------------------------------+   |
```

```
|   +--------------------------------+   |
|   |  Mot de passe          []     |   |
|   +--------------------------------+   |
|                                        |
|        [ SE CONNECTER ]                |
|                                        |
|    Message d'erreur (si erreur)       |
+----------------------------------------+
```

4.3 Dashboard Principal

```
+----------------------------------------+
| Football Manager 5v5 -- Bienvenue, admin |
+----------+----------+----------+--------+
| Terrains | Resas    |Utilisat. | Total  |
| actifs   | du jour  | actifs   | resas  |
|   5      |    3     |   12     |  18    |
+----------┴----------┴----------┴--------+
| [Dashboard][Users][Terrains][Resas][Cal]|
|    Onglets de navigation par rôle       |
+----------------------------------------+
```

4.4 Gestion des Réservations

```
Filtres : [Date       v] [Statut v] [Terrain v]
+----+----------+----------+--------------+-----------+----------+
| ID | Client   | Terrain  | Date/Heure   | Statut    | Actions  |
+----+----------+----------+--------------+-----------+----------+
|  1 | user1    | Terrain A| 30/03 14h-16h|  Attente|[][][]|
|  2 | test2    | Terrain B| 30/03 18h-20h|  Confirmé|[][]  |
|  3 | user1    | Terrain C| 31/03 10h-12h|  Annulé |[]      |
+----┴----------┴----------┴--------------┴-----------┴----------+
```

---

5. Sécurité

5.1 Authentification

- Vérification du hash bcrypt : bcrypt.checkpw(password.encode(), hash.encode())
- Rate limiting : blocage automatique après 5 tentatives échouées en moins de 10 minutes
- Déconnexion propre avec nettoyage de la session en mémoire

5.2 Contrôle d'Accès RBAC

```
# Vérification des droits dans les controllers
def require_role(user, required_roles):
    if user.role not in required_roles:
        raise PermissionError("Accès non autorisé")

# Exemple d'utilisation
def delete_user(current_user, user_id):
    require_role(current_user, ['superadmin'])
    # ... suppression
```

5.3 Validation des Données

- Côté application : validation Python avant toute écriture en BDD
- Côté BDD : contraintes SQL (UNIQUE, FK, ENUM, UNIQUE KEY anti-conflit)
- Détection de conflit : requête SQL + validation Python en double

5.4 Sécurité des Mots de Passe

```
# Création (hashing)
import bcrypt
password_hash = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt(rounds=12))

# Vérification (login)
bcrypt.checkpw(password.encode('utf-8'), stored_hash.encode('utf-8'))
```

---

6. Tests

6.1 Résultats Globaux

Catégorie	Résultat
Tests unitaires	124/124 passent [OK]
Tests d'intégration	18/18 réussis [OK]
Tests fonctionnels	15/15 scénarios validés [OK]
Couverture globale	**87%**
Crash en 48h de test	0 [OK]

6.2 Tests Unitaires (exemples)

```
class TestUserModel(unittest.TestCase):
    def test_password_hashing(self):
        # le hash doit être différent du mot de passe en clair
        hash = bcrypt.hashpw("admin123".encode(), bcrypt.gensalt(12))
        self.assertNotEqual(hash, "admin123".encode())
        self.assertTrue(bcrypt.checkpw("admin123".encode(), hash))

    def test_user_validation(self):
        # username trop court → ValueError
        with self.assertRaises(ValueError):
            create_user(username="ab", email="test@test.com")

class TestReservationModel(unittest.TestCase):
    def test_time_validation(self):
        # end avant start → erreur
        with self.assertRaises(ValueError):
            create_reservation(start="14:00", end="13:00")

    def test_conflict_detection(self):
        # deux réservations sur le même terrain/créneau
        self.assertTrue(reservation1.conflicts_with(reservation2))
```

6.3 Couverture par Module

Module	Couverture
models/user.py	95%
models/terrain.py	92%
models/reservation.py	88%
controllers/	85%
views/	78%
utils/	90%
TOTAL	**87%**

6.4 Tests de Performance

Opération	Cible	Mesuré	Statut
Connexion (login)	< 1s	0.3s	[OK]
Chargement dashboard	< 2s	1.2s	[OK]
Création réservation	< 2s	0.8s	[OK]
Requête conflits	< 1s	0.4s	[OK]
Rafraîchissement calendrier	< 1s	0.6s	[OK]

---

7. Installation et Démarrage

Prérequis

- Python 3.8+
- XAMPP avec MySQL (base foot5 initialisée)
- Droits d'écriture sur le dossier d'installation

Installation

```
# 1. Cloner le dépôt
git clone https://github.com/warkozz/projet-ppe-exam.git
cd projet-ppe-exam/logiciel-gestion/desktop_app
```

```
# 2. Créer l'environnement virtuel
python -m venv .venv
.venv\Scripts\activate         # Windows

# 3. Installer les dépendances
pip install -r requirements.txt

# 4. Initialiser la base de données
# Exécuter schema_mysql.sql puis seed_data_mysql_fixed.sql dans phpMyAdmin (XAMPP)

# 5. Lancer l'application
python hybrid_main.py
```

Comptes par défaut

Compte	Mot de passe	Rôle
admin	admin123	Superadmin
manager	manager123	Admin (Gestionnaire)
user1	user123	Utilisateur

Hakim Rayane -- BTS SIO SLAM -- PPE Foot5 -- Mars 2026

DOC 05

Code Source -- Liens GitHub

Projet PPE Foot5 -- BTS SIO SLAM

Étudiant : Hakim Rayane

Date : Mars 2026

Liens vers les Dépôts

Application Légère -- Web Client

URL : https://github.com/warkozz/projet-ppe-foot5-web

Branche principale : develop

Langage dominant : TypeScript (77%) + Python (20%)

Application Lourde -- Admin Desktop

URL : https://github.com/warkozz/projet-ppe-exam

Branche principale : main

Langage dominant : Python 100%

Structure des Dépôts

Appli Légère (projet-ppe-foot5-web)

```
projet-ppe-foot5-web/
|
+-- backend/                        # API FastAPI (Python)
|   +-- main.py                     # Point d'entrée
|   +-- app/
|   |   +-- models/                 # Modèles SQLAlchemy
|   |   |   +-- user.py
```

```
|   |   |   +-- terrain.py
|   |   |   +-- reservation.py
|   |   +-- routes/                 # Endpoints REST
|   |   |   +-- auth.py
|   |   |   +-- terrains.py
|   |   |   +-- reservations.py
|   |   +-- schemas/                # Validation Pydantic
|   |   +-- services/               # Logique métier
|   |   +-- utils/                  # JWT, hashing, config
|   +-- test_api.py                 # Tests fonctionnels API
|   +-- test_logic.py               # Tests logique métier
|   +-- requirements.txt
|
+-- frontend/                       # App React (TypeScript)
|   +-- src/
|       +-- pages/                  # Pages de l'application
|       |   +-- Home.tsx
|       |   +-- Login.tsx
|       |   +-- Register.tsx
|       |   +-- Terrains.tsx
|       |   +-- Reservation.tsx
|       |   +-- MonEspace.tsx
|       |   +-- Profile.tsx
|       +-- components/             # Composants réutilisables
|       +-- contexts/               # AuthContext
|       +-- services/               # api.ts (Axios)
|       +-- types/                  # Interfaces TypeScript
|
+-- docs/                           # Documentation projet
    +-- BACKEND_API.md              # Doc technique API complète
    +-- BACKEND_STATUS.md           # État des fonctionnalités
    +-- PROJECT_HISTORY.md          # Historique du développement
```

Appli Lourde (projet-ppe-exam)

```
projet-ppe-exam/
|
+-- logiciel-gestion/
|   +-- database/                   # Scripts SQL
|   |   +-- schema_mysql.sql        # Schéma complet
|   |   +-- seed_data_mysql_fixed.sql# Données de test
|   |   +-- verify_install.sql      # Script de vérification
|   |
|   +-- desktop_app/                # Application Python/PySide6
|   |   +-- hybrid_main.py          # Point d'entrée
|   |   +-- requirements.txt
|   |   +-- app/
|   |   |   +-- config.py           # Configuration
|   |   |   +-- models/             # Modèles SQLAlchemy
```

```
|   |   |   |   +-- db.py
|   |   |   |   +-- user.py
|   |   |   |   +-- terrain.py
|   |   |   |   +-- reservation.py
|   |   |   +-- controllers/         # Logique métier
|   |   |   |   +-- auth_controller.py
|   |   |   |   +-- user_controller.py
|   |   |   |   +-- terrain_controller.py
|   |   |   |   +-- reservation_controller.py
|   |   |   +-- views/               # Interfaces PySide6
|   |   |   |   +-- login_view.py
|   |   |   |   +-- hybrid/
|   |   |   |       +-- dashboard_view.py
|   |   |   |       +-- user_view.py
|   |   |   |       +-- terrain_view.py
|   |   |   |       +-- reservation_view.py
|   |   |   |       +-- calendar_view.py
|   |   |   +-- services/            # Services (calendrier, etc.)
|   |   |   +-- styles/              # Thème Material Design
|   |   |   +-- utils/               # hashing bcrypt
|   |   +-- run.bat                  # Script de démarrage Windows
|   |
|   +-- documentation/
|       +-- technical_documentation.md
|       +-- user_manual.md
|
+-- PPE-ANNEXES/                     # Documents PPE (étapes 1-10)
    +-- 01_Introduction_Projet.md
    +-- 02_Maquettes_UI.md
    +-- 03_Cahier_Charges.md
    +-- 04_Conception_BDD/
    +-- 05_Planification.md
    +-- 06_Developpement.md
    +-- 07_Tests.md
    +-- 08_Documentation.md
    +-- 09_Rendu_Final.md
    +-- 10_Evaluation.md
```

Métriques du Code

Appli Lourde (Desktop)

Métrique	Valeur
Lignes de code Python	~9 000 lignes
Nombre de modules	15+
Nombre de classes	25+
Méthodes documentées	100+
Conformité PEP8	[OK]
Type hints	95%
Couverture de tests	87%

Appli Légère (Web)

Métrique	Valeur
Langage principal	TypeScript 77%
Backend Python	20%
Endpoints API	20+
Tests API	pytest + requests
Documentation	Swagger auto-générée

Cloner et Tester

Appli Légère

```
git clone https://github.com/warkozz/projet-ppe-foot5-web.git
cd projet-ppe-foot5-web/backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
# → Swagger : http://localhost:8000/docs

cd ../frontend
npm install ; npm start
# → App React : http://localhost:3000
```

Appli Lourde

```
git clone https://github.com/warkozz/projet-ppe-exam.git
cd projet-ppe-exam/logiciel-gestion/desktop_app
pip install -r requirements.txt
python hybrid_main.py
```

Informations Techniques Clés

Élément	Valeur
Auteur	Hakim Rayane (warkozz)
Profil GitHub	https://github.com/warkozz
Formation	BTS SIO SLAM
Période de développement	Décembre 2025 - Mars 2026
Base de données	MySQL foot5 -- XAMPP
Système d'exploitation	Windows

Hakim Rayane -- BTS SIO SLAM -- PPE Foot5 -- Mars 2026

Fin du Dossier Technique

Projet PPE -- BTS SIO SLAM

Hakim Rayane -- Mars 2026

App legere (Web) : github.com/warkozz/projet-ppe-foot5-web

App lourde (Admin) : github.com/warkozz/projet-ppe-exam